Meeting Room Booking API — Assignment Report

1. Introduction

This assignment implements and deploys a meeting-room booking REST API. The application is built with FastAPI and provides authentication, room information, and booking operations. It uses an embedded H2 database for application data and is containerized with Docker. Kubernetes resources run the application, while Traefik routes browser traffic to the application Service through an Ingress.

The project also demonstrates installing Helm-managed services, including Traefik as the Ingress controller and NGINX as a separate web server.

2. Objectives

- Implement a REST API for authenticated meeting-room booking.
- Package and run the API as a Docker container.
- Deploy the container to a local Docker Desktop Kubernetes cluster.
- Use Kubernetes Services and Ingress to route HTTP traffic.
- Use Helm to install Traefik and optionally NGINX.
- Observe workloads, pod status, restarts, images, and Kubernetes events.
- Preserve database files using persistent volume claims.

3. Technologies Used

Technology	Purpose
Python 3.11	Application runtime
FastAPI	REST API and interactive OpenAPI documentation
SQLAlchemy	Database ORM
H2	Embedded relational database
Docker	Container image build and execution
Docker Desktop Kubernetes	Local Kubernetes cluster
Kubernetes	Workload orchestration, Services, storage, and Ingress resources
Helm	Kubernetes package manager
Traefik	Ingress controller and HTTP traffic router
NGINX	Optional web server deployed separately using Helm

The recorded Helm version was `v4.3.1`. The application image used by the StatefulSet is `docker-demo:v1`.

4. Application Overview

The application exposes endpoints for:

- Logging in and obtaining a JWT bearer token.
- Viewing the authenticated user's profile.
- Listing available rooms and retrieving an individual room.
- Creating and viewing room bookings.
- Listing a user's bookings and cancelling an owned booking.

Room inventory is read from `rooms.json`. The API documentation is available at `/docs`. The root path `/` redirects to `/docs`.

The H2 database is embedded, so no database server password is configured. `SECRET_KEY` is a JWT signing secret; it is stored as a Kubernetes Secret and is separate from database credentials.

5. Deployment Architecture

```
Browser (http://localhost)
        |
        v
Traefik Ingress Controller
        |
        v
Ingress rule: host localhost, path /
        |
        v
ClusterIP Service: docker-demo:80
        |
        +------> StatefulSet pod: docker-demo-0
        |           Persistent volume claim for H2 data
        |
        +------> StatefulSet pod: docker-demo-1
                    Persistent volume claim for H2 data
```

The StatefulSet is named `docker-demo` and is configured with two replicas. Each replica has its own persistent volume claim mounted at `/app/data`. Persistent claims can preserve data when a pod is replaced, but the two H2 databases are separate and do not synchronize booking data.

NGINX was installed as an optional separate web server. It is not the Ingress controller: Traefik remains responsible for routing. The Ingress backend can be switched between the `nginx` Service and the `docker-demo` Service.

6. Installation and Deployment Procedure

The following commands summarize the Docker Desktop setup. Run them in PowerShell from the `meeting-room-booking` project directory where applicable.

6.1 Verify the local tools and Kubernetes cluster

Install Docker Desktop using the official Windows installation guide, start Docker Desktop, and enable its Kubernetes cluster.

```
docker --version
docker compose version
kubectl config current-context
kubectl get nodes
helm version
```

The Kubernetes node should show `Ready` before deploying workloads.

6.2 Install Traefik with Helm

```
helm repo add traefik https://traefik.github.io/charts
helm repo update
helm upgrade --install traefik traefik/traefik `
  --namespace traefik --create-namespace
kubectl get pods,services -n traefik
```

Traefik watches Ingress resources and forwards matching HTTP requests to Kubernetes Services.

6.3 Build the application image and create its JWT Secret

```
docker build -t docker-demo:v1 .
$jwtSecret = python -c "import secrets; print(secrets.token_urlsafe(32))"
kubectl create secret generic meeting-room-booking-secrets `
  --from-literal="secret-key=$jwtSecret"
```

The generated JWT secret should be kept private and should not be committed to source control.

6.4 Apply the Kubernetes resources

```
kubectl apply -f configmap.yaml
kubectl apply -f service.yaml
kubectl apply -f deployment.yaml
kubectl apply -f ingress.yaml
kubectl rollout status statefulset/docker-demo
kubectl get pods,services,ingress,pvc
```

The resource named `docker-demo` is a Kubernetes **StatefulSet**, despite the informal use of “deployment” to refer to deploying the application. The StatefulSet uses the `docker-demo:v1` image, readiness and liveness probes, and two replicas.

6.5 Optionally install NGINX with Helm

```
helm upgrade --install nginx `
  oci://registry-1.docker.io/bitnamicharts/nginx `
  --namespace default
kubectl get pods,services -n default
```

NGINX is an optional web server. To serve it at `http://localhost/`, the Ingress backend must point to the `nginx` Service. To serve the meeting-room API again, the backend must point to `docker-demo`. Apply the edited Ingress manifest after changing the backend:

```
kubectl apply -f ingress.yaml
```

7. Verification and Observations

The following commands were used to inspect and verify the deployment:

```
kubectl get nodes
kubectl get pods -l app=docker-demo
kubectl describe pod <pod-name>
kubectl get svc,endpoints
kubectl describe ingress meeting-room
curl.exe -i http://localhost/
```

The observed cluster node was `Ready`. Two `docker-demo` pods were configured by the StatefulSet. The app Service selected the app pods, and the Ingress matched the `localhost` host rule. The application was reached at `http://localhost/docs`; after adding the root redirect, `http://localhost/` redirected to `/docs`.

During the NGINX routing demonstration, an Ingress backend typo (`ngnix` instead of `nginx`) caused a routing failure. Correcting the Service name and applying the Ingress restored routing. This demonstrated that the Ingress backend must reference an existing Service in the same namespace.

The pod inspection fields have the following meanings:

- **Status:** Pod lifecycle phase and readiness state.
- **Restart count:** Number of container restarts; repeated increases can

indicate a crash or failed health check.

- **Image:** Container image configured for that pod, here `docker-demo:v1`.
- **Events:** Kubernetes messages about scheduling, image pulls, volumes,

probes, and container startup.

8. Persistence and Security

- H2 is embedded; no database password is used in this configuration.
- The JWT `SECRET_KEY` is kept in a Kubernetes Secret, not in the container

image.

- Each StatefulSet replica receives a separate 1 GiB persistent volume claim.
- A replica can reconnect to its claim after pod replacement, subject to the

cluster's storage behavior and claim retention.

- The two replicas have separate H2 files. A shared database service is

required if all replicas must operate on the same booking data.

- The local Ingress exposes the application through the local Docker Desktop

cluster; this setup does not by itself publish the app to the public internet.

9. Screenshots

10. Conclusion

The assignment demonstrates the end-to-end deployment of a REST API with Docker and Kubernetes. Helm was used to install Traefik, which routes requests through an Ingress to the meeting-room application's Service. The app ran as a two-replica StatefulSet with per-pod persistent storage, and NGINX was deployed separately to demonstrate changing the Ingress backend.

The exercise also highlights important operational considerations: verify Service names in Ingress rules, distinguish StatefulSets from Deployments, protect JWT signing secrets, and use shared database infrastructure when multiple replicas need consistent application data. s